

Operator Overloading & File Handling

A comprehensive guide to mastering unary operators, binary operators, comparison operators, arithmetic assignment operators, and file handling in object-oriented programming. This module covers Lectures 27 through 30, building from foundational operator concepts to practical file I/O operations.

W14-W15

LECTURES 27-30

Introduction to Operator Overloading

Operator overloading allows programmers to redefine the behavior of standard operators (like `+`, `-`, `*`, `==`) when used with user-defined types such as classes. This feature enables objects to interact using familiar, intuitive syntax rather than verbose method calls. For example, instead of writing `obj1.add(obj2)`, you can simply write `obj1 + obj2`, making code more readable and expressive. This module progresses from unary operators (L27) through binary and comparison operators (L28), arithmetic assignment operators (L29), and concludes with file handling operations (L29–L30).

Unary Operators

Operators that act on a single operand, such as `++`, `--`, and unary `-`. Covered in Lecture 27.

Binary Operators

Operators that work with two operands, including arithmetic, string concatenation, and comparison. Covered in Lecture 28.

Arithmetic Assignment

Compound operators like `+=`, `-=`, `*=` that combine an operation with assignment. Covered in Lecture 29.

File Handling

Reading, writing, appending, and deleting records from files using overloaded operators and file streams. Covered in Lectures 29–30.

Lecture 27: Unary Operators

W13 · L27

Unary operators operate on a **single operand**. In C++, you can overload unary operators such as increment (++), decrement (--), unary minus (-), logical NOT (!), and dereference (*) for your own classes. Overloading these operators allows objects to behave like built-in types, supporting intuitive syntax.

Operator Arguments

When overloading a unary operator as a **member function**, it takes **no parameters** (for prefix) or one dummy `int` parameter (for postfix). As a **non-member (friend) function**, it takes one parameter — the object being operated on.

- Prefix ++obj: no parameters
- Postfix obj++: one `int` dummy parameter
- Unary -obj: no parameters (member)
- Logical !obj: no parameters (member)

Operator Return Values

The return type of a unary operator depends on the operator and whether it is prefix or postfix. Prefix operators typically return a **reference** to the modified object (`MyClass&`), allowing chaining. Postfix operators return a **copy** of the original value (`MyClass`) before modification, since the original must be preserved for the return.

- Prefix ++: returns `MyClass&` (reference)
- Postfix ++: returns `MyClass` (by value)
- Unary -: returns a new modified object
- Return by reference enables expressions like ++++obj

Prefix vs. Postfix: Key Distinctions

W13 · L27

One of the most important distinctions in unary operator overloading is between **prefix** and **postfix** forms of `++` and `--`. The compiler differentiates them by the presence of a dummy `int` parameter in the postfix version. Understanding this distinction is critical for writing correct and efficient overloaded operators.

Prefix Increment (`++obj`)

The prefix form increments the object **first**, then returns a reference to the modified object. Signature: `MyClass& operator++()`. Because it returns a reference, it supports chaining and is generally more efficient (no temporary copy created).

Postfix Increment (`obj++`)

The postfix form saves a copy of the original value, increments the object, then returns the **original** (unmodified) copy. Signature: `MyClass operator++(int)`. The `int` parameter is a dummy — it is never used, it only signals to the compiler that this is the postfix version. This form is less efficient due to the temporary copy.

Best Practice

Prefer prefix increment in your own code when the return value is not needed, as it avoids the overhead of creating a temporary copy. Only use postfix when you specifically need the original value before incrementing. Always return `*this` by reference for prefix, and return the saved copy by value for postfix.

Lecture 28: Overloading Binary Operators

W14 · L28

Binary operators require **two operands** and include arithmetic operators (+, -, *, /), relational operators, and string concatenation. Overloading binary operators enables objects to participate in natural mathematical and logical expressions.



Arithmetic Operators

Overload +, -, *, / to perform custom arithmetic on objects. For example, adding two `Vector` objects or two `Complex` numbers. Can be implemented as member functions (left operand is `*this`) or friend functions (both operands are parameters).



Concatenating Strings

The + operator can be overloaded to concatenate string objects. When overloading for string concatenation, ensure proper memory management — allocate new memory for the result and handle self-assignment. Return a new string object by value.



Multiple Overloading

A single operator can be overloaded in multiple ways — as both a member and a non-member function, or for different parameter types. This enables flexible usage patterns, such as supporting both `obj + 5` and `5 + obj` through separate overloads.

Comparison Operators

W14 · L28

Comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) allow objects to be compared for equality or ordering. Overloading these is essential for using custom types in sorted containers, algorithms, and conditional expressions.

Equality Operators

The `==` and `!=` operators check whether two objects are logically equivalent. Typically, `==` is implemented first, and `!=` simply returns `!(a == b)`. Equality should compare all relevant member variables. Return type is `bool`.

- Compare all significant data members
- Return `true` if all members match
- `!=` can be derived from `==`
- Essential for use in `std::find`, `std::sort`

Relational Operators

Relational operators define ordering between objects. At minimum, overload `<` — the others can often be expressed in terms of it. Critical for sorting algorithms and ordered containers like `std::map` and `std::set`.

- `<` and `>`: define ordering
- `<=` and `>=`: include equality
- Return `bool` values
- Must define a consistent strict weak ordering
- Used by `std::sort`, `std::lower_bound`

📌 When overloading comparison operators, ensure **consistency**: if `a == b` is true, then `a < b` and `b < a` must both be false. Violating this leads to undefined behavior in sorted containers.

Lecture 29: Arithmetic Assignment Operators

W14 · L29

Arithmetic assignment operators (`+=`, `-=`, `*=`, `/=`) combine an arithmetic operation with assignment. They modify the left-hand operand in place and return a reference to it, enabling chaining like `a += b += c`.

1

Implementation as Member Functions

Arithmetic assignment operators are almost always implemented as **member functions** because they modify the left-hand operand (`*this`). They take one parameter (the right-hand operand) and return `*this` by reference to support chaining.

2

Relationship with Binary Operators

A common pattern is to implement `+=` first, then define `+` in terms of it: `operator+(const MyClass& rhs) { MyClass temp = *this; temp += rhs; return temp; }`. This reduces code duplication and ensures consistent behavior between the two operators.

3

Return by Reference

Unlike binary operators that return by value, assignment operators return `MyClass&` (a reference to `*this`). This enables chained expressions and matches the behavior of built-in types. Never return by value — it creates an unnecessary copy and breaks chaining.

4

Self-Assignment Safety

While less critical for `+=` than for the copy assignment operator (`=`), it is still good practice to handle self-assignment gracefully. In most cases, self-assignment with `+=` is well-defined (doubling the value), but explicit checks can prevent edge-case bugs.

Lecture 29: Introduction to File Handling

W14 · L29

File handling allows programs to persist data beyond execution by reading from and writing to files on disk. In C++, the `<fstream>` library provides three main classes: `ifstream` (input file stream), `ofstream` (output file stream), and `fstream` (bidirectional file stream).



Opening Files

Files are opened using the `open()` method or via the constructor. Always check if the file opened successfully using `is_open()` before proceeding. Specify the file mode: `ios::in`, `ios::out`, `ios::app`, `ios::binary`, etc.



Closing Files

Always close files explicitly using `close()` when done. While destructors will close files automatically, explicit closing ensures data is flushed and resources are released promptly. Failing to close files can result in data loss or resource leaks.



Error Handling

Check stream state using `fail()`, `bad()`, and `eof()` to detect errors during file operations. Common issues include file not found, permission denied, disk full, and read/write failures. Handle these gracefully to prevent crashes.

Lecture 30: Writing & Reading Data

W14 · L30

Once a file is opened, data can be written to it or read from it using stream operators and member functions. Understanding the difference between text and binary modes, and knowing how to format and parse data correctly, is essential for robust file handling.

Writing Data to Files

Use `ofstream` or `fstream` with `ios::out` to write data. The insertion operator `<<` works similarly to `cout`. For binary data, use `write()` with `reinterpret_cast<char*>`.

- Text mode: `outFile << "Hello" << value;`
- Binary mode: `outFile.write((char*)&obj, sizeof(obj));`
- Use `endl` or `\n` for newlines
- Overwrites existing file by default

Reading Data from Files

Use `ifstream` or `fstream` with `ios::in` to read data. The extraction operator `>>` works like `cin`. For reading full lines, use `getline()`. For binary data, use `read()`.

- Text mode: `inFile >> value;`
- Full lines: `getline(inFile, line);`
- Binary mode: `inFile.read((char*)&obj, sizeof(obj));`
- Check `eof()` to detect end of file
- Loop with `while(inFile >> value)` for robust reading

Lecture 30: Appending Files & Deleting Records

W14 · L30

Advanced file operations include appending data to existing files without overwriting them, and deleting specific records from a file. These operations require careful handling of file positions and often involve creating temporary files.

Appending Files

To append data to an existing file, open it with the `ios::app` flag: `ofstream outFile("data.txt", ios::app);`. This positions the write pointer at the end of the file, so new data is added without overwriting existing content. Alternatively, `ios::ate` opens the file and positions at the end, but allows seeking to other positions. Appending is commonly used for log files and transaction records.

Deleting Records

C++ does not provide a direct "delete record" operation for files. The standard approach is to: **(1)** Open the original file for reading. **(2)** Open a temporary file for writing. **(3)** Copy all records *except* the one(s) to delete into the temporary file. **(4)** Close both files. **(5)** Delete the original file and rename the temporary file to the original name. This ensures data integrity even if the operation is interrupted.

Key Takeaways

Operator overloading and file handling are powerful tools that, when combined, allow you to create classes that behave like built-in types while persisting their state to disk. Always validate file operations, handle errors gracefully, close files properly, and use the appropriate file mode (`in`, `out`, `app`, `binary`) for each task. Practice overloading operators in conjunction with file I/O to build robust, reusable classes.

Copy Non-Matches

Write records not matching target to temp.

Close Files

Close both original and temporary files.

Open Original

Open the original file for reading.

Replace Original

Replace original file with the temp file.

The four-step record deletion process ensures data safety by never modifying the original file directly — all changes go through a temporary file first, preventing data loss if an error occurs mid-operation.